

# Database Management – SQL & PL/SQL

## Section A: Concept Application

### 1. Explain how relational databases help maintain accuracy in expense records.

I have 2 table user and expenses if I use expenses as a forenkey relation many to many  
I found correct expenses by user

| Expense_id | User_d | amount |
|------------|--------|--------|
| 101        | 1      | 500    |

### 2. Why are constraints important in personal finance data?

In finance data accuracy is very important and constraints is a rools in db  
Example (NOT NULL , PRIMARY KEY , DONOT NSER DATA USER NOT EXIST)

- amount DECIMAL(10,2) NOT NULL
- user\_id INT PRIMARY KEY

### 3. How does GROUP BY help analyze spending patterns?

If I have product and amount and user bay two same product  
Then amount add both product and show single entry

| category | amount |
|----------|--------|
| Food     | 200    |
| Food     | 300    |
| Rent     | 5000   |

```
SELECT category,  
SUM(amount)  
FROM expenses  
GROUP BY category;
```

**category total**

|      |      |
|------|------|
| Food | 500  |
| Rent | 5000 |

**4. Explain a scenario where rollback is required during expense entry.**

If product amount 500 and user by mistake enter 50000 then and developer  
Add condition if amount grater than amount ,amount do rollback  
Amount going to undo , transaction canceled

```
if(amount > 10000){  
rollback  
}
```

**5. How do views help users track monthly expenses efficiently?**

If user generate view , view help for access data according to expenses date  
And access monthly data

```
CREATE VIEW MonthlyExpenses AS  
SELECT *  
FROM expenses  
WHERE MONTH(expense_date)=5;
```

**6. Why use triggers for automatic category or balance updates?**

If user perform event triggers call automatic if developer user triggers in update  
Event and pass query like old data save to other table and new data save to user table  
When user perform update event old data was save auto to another table , don't do that  
Task manual

## Section B: SQL Hands-On

### ***Given Database Schema (DO NOT MODIFY)***

```
users ( user_id INT PRIMARY KEY,  
        name VARCHAR(50),  
        email VARCHAR(100),  
        created_at DATE );
```

```
categories ( category_id INT PRIMARY KEY,  
             category_name VARCHAR(50) );
```

```
expenses ( expense_id INT PRIMARY KEY,  
           user_id INT,  
           category_id INT,  
           amount DECIMAL(10,2),  
           expense_date DATE,  
           FOREIGN KEY (user_id) REFERENCES users(user_id),  
           FOREIGN KEY (category_id) REFERENCES categories(category_id)  
           );
```

### **1. DDL Understanding**

Without modifying the schema:

- **Explain why foreign keys are used:** Specifically, why must user\_id in the expenses table correspond to a real ID in the users table?

- **Mention one issue that would occur if foreign keys were removed:** Describe the risk of "Orphaned Records" (e.g., an expense existing for a user who has been deleted)

Foreign key used for according to user id find expense

If not use foreign key how to find out that expense is which user

That means user is not exist but expense is exist

### **2. DML Operations**

Write SQL queries to:

- **Insert 5 users into the users table with unique names and emails.**

```
INSERT INTO users VALUES  
(1,'Arbaz','arbaz@gmail.com','2026-01-01'),  
(2,'Aman','aman@gmail.com','2026-01-02'),  
(3,'Rahul','rahul@gmail.com','2026-01-03'),  
(4,'Ali','ali@gmail.com','2026-01-04'),
```

```
(5,'Zaid','zaid@gmail.com','2026-01-05');
```

- **Insert 3 categories (e.g., 'Food', 'Rent', 'Entertainment').**

```
INSERT INTO categories VALUES  
(1,'Food'),  
(2,'Rent'),  
(3,'Entertainment');
```

- **Insert 10 expense records distributed across different users and categories.**

```
INSERT INTO expenses VALUES  
(1,1,1,250,'2026-05-01'),  
(2,1,2,5000,'2026-05-02'),  
(3,2,1,300,'2026-05-03'),  
(4,2,3,800,'2026-05-04'),  
(5,3,2,4500,'2026-05-05'),  
(6,3,1,200,'2026-05-06'),  
(7,4,3,1200,'2026-05-07'),  
(8,4,1,350,'2026-05-08'),  
(9,5,2,6000,'2026-05-09'),  
(10,5,3,1500,'2026-05-10');
```

- **Update one incorrect expense: Use the UPDATE command to change the amount of a specific expense\_id.**

```
UPDATE expenses  
SET amount = 500  
WHERE expense_id = 2;
```

- **Delete one expense: Remove a record where the amount is less than a specific value**

```
DELETE FROM expenses  
WHERE amount < 250;
```

### 3. Data Retrieval

Write queries to:

- **Display all expenses with details: Show the expense\_date, amount, name (from users), and category\_name (from categories) using INNER JOIN.**

```
SELECT  
e.expense_date,  
e.amount,  
u.name,  
c.category_name  
  
FROM expenses e  
  
INNER JOIN users u  
ON e.user_id = u.user_id
```

```
INNER JOIN categories c
ON e.category_id = c.category_id;
```

- **Show total expense amount per category: Use SUM(amount) and GROUP BY category\_name.**

```
SELECT
c.category_name,
SUM(e.amount) AS total_expense
```

```
FROM expenses e
```

```
INNER JOIN categories c
ON e.category_id = c.category_id
```

```
GROUP BY c.category_name;
```

- **Display users sorted by total spending: List user names and their total sum of expenses, ordered from highest to lowest**

```
SELECT
u.name,
SUM(e.amount) AS total_spending
```

```
FROM users u
```

```
INNER JOIN expenses e
ON u.user_id = e.user_id
```

```
GROUP BY u.name
```

```
ORDER BY total_spending DESC;
```

#### 4. Views

- **Create a view named ActiveUsersView: This view should list the name and email of any user who has logged more than 5 individual expense records.**

```
CREATE VIEW ActiveUsersView AS
```

```
SELECT
```

```
u.name,
```

```
u.email
```

```
FROM users u
```

JOIN expenses e

ON u.id = e.user\_id

GROUP BY u.id, u.name, u.email

HAVING COUNT(e.id) > 5;

• **Query the view: Write a simple SELECT statement to show all data from ActiveUsersView.**

SELECT \* FROM ActiveUsersView;